🚀 GOOGLE DEEPMIND ADVANCED AGENTIC CODING

Engenharia de Agentes com Google Antigravity (AGY)

Do Zero ao Nível Expert: Pair Programming, Slash Commands, Arquitetura de Skills, Protocolo MCP, Subagentes Paralelos e SDK em Python

Autoridade Técnica: Google DeepMind Team

Edição: Completa & Exaustiva (2026)

Escopo: Antigravity CLI, IDE, App 2.0 & SDK

Formato: Manual Técnico de Treinamento

📑 Estrutura Geral do Curso Masterclass

Capítulo 1: Fundamentos da Revolução Agentic Coding & Superfícies AGY

Capítulo 2: O Protocolo de Pair Programming & Filosofia de Desenvolvimento

Capítulo 3: Manual Exaustivo de Slash Commands & Modos Avançados

Capítulo 4: O Sistema de Customizações: Rules, Skills, Plugins & Sidecars

Capítulo 5: Model Context Protocol (MCP) & Inspeção em Tempo de Execução

Capítulo 6: Arquitetura Multi-Agente: Orquestração Paralela de Subagentes

Capítulo 7: Desenvolvimento Autônomo com o Antigravity SDK em Python

Capítulo 8: Troubleshooting Avançado, Prevenção de Perda de Dados & Checklist

Capítulo 1: Fundamentos da Revolução Agentic Coding & Superfícies AGY

O **Google Antigravity (AGY)** representa a transição paradigmática entre a geração passiva de código (chatbots tradicionais) e a **Engenharia de Agentes Autônomos (Agentic Coding)**. Em vez de simplesmente responder a perguntas com trechos isolados de texto, o Antigravity opera diretamente no seu ambiente local, com capacidade de ler arquivos, modificar repositórios, executar comandos no terminal do SO, debugar erros e auditar interfaces em tempo real.

> 💡 **O Conceito do Agentic Loop**
>
> O agente opera em um ciclo contínuo de 5 etapas: **Percepção** (leitura de arquivos e logs) ➔ **Decisão** (planejamento técnico) ➔ **Execução** (edição de código / ferramentas) ➔ **Verificação** (build / testes) ➔ **Ajuste** (correção de falhas).

1.1 As 3 Superfícies do Google Antigravity

Superfície	Descrição & Caso de Uso Principal	Recursos Principais
Antigravity CLI (`agy`)	Interface em linha de comando ultrarrápida para terminais, servidores e scripts autônomos.	Integração direta com o shell do SO, execução de tarefas assíncronas e scripts de CI/CD.
Antigravity IDE	Ambiente de desenvolvimento integrado AI-First com lentes de código em tempo real.	Sugestões contextuais inline, chat lateral sincronizado com o editor e atalhos rápidos.
Antigravity App 2.0	Aplicação desktop com painel auxiliar HTML para gestão visual de agentes.	Painel de Subagentes, visualizador de diffs, monitor de tarefas em segundo plano e múltiplos terminais.

Capítulo 2: O Protocolo de Pair Programming & Filosofia de Desenvolvimento

O Pair Programming com o Antigravity obedece a um conjunto de regras rígidas criadas para garantir máxima confiabilidade, prevenção de regressões e código de qualidade de produção.

> ⚠️ **As 5 Leis Invioláveis do Antigravity**
>
> 1. **Nunca Adivinhe o Código:** Nunca infira assinaturas de métodos, esquemas de banco ou rotas sem inspecionar o arquivo fonte original.
> 2. **Diagnóstico Baseado em Evidências:** Nunca forme uma hipótese de erro sem ler a pilha de erro completa e não truncada.
> 3. **Proibição de Soluções Superficiais:** É proibido mascarar erros com tratamentos silenciosos, retornos nulos falsos ou comentar asserções de teste quebradas.
> 4. **Verificação Autônoma Obrigatória:** Nunca declare um trabalho como concluído sem executar os comandos de build e testes (`npx tsc` , `npm run build` , `pytest`).
> 5. **Manutenção de Contratos de API:** Edições em assinaturas de funções exigem a atualização de todos os pontos de invocação no repositório.

2.1 Gestão de Artefatos de Desenvolvimento

O Antigravity utiliza dois documentos dinâmicos para alinhar o progresso com o desenvolvedor:

- `implementation_plan.md` : Plano de implementação detalhado com descrição de metas, análise de risco, lista de arquivos alterados/criados e plano de verificação.
- `walkthrough.md` : Guia final demonstrando as alterações concluídas, testes executados e capturas/provas do resultado.

Capítulo 3: Manual Exaustivo de Slash Commands & Modos Avançados

Os **Slash Commands** permitem acionar fluxos de trabalho avançados diretamente no chat:

3.1 Tabela Geral de Slash Commands

Comando	Modo de Operação	Quando Utilizar
`/grill-me`	Entrevista Interativa de Arquitetura	Use no início de grandes tarefas para definir decisões de design e ambiguidades antes de gerar código.
`/goal`	Execução Autônoma Rígida (Overnight)	Use para metas longas e complexas. O agente trabalhará de forma contínua até alcançar a validação total.
`/learn`	Persistência de Aprendizado	Use após corrigir o agente ou alinhar uma regra personalizada para que o padrão fique gravado para sessões futuras.
`/schedule`	Temporizadores e Cron Jobs	Use para agendar lembretes ou checagens periódicas em segundo plano.
`/browser`	Navegação & Testes Visuais	Use para abrir e automotivar interações no navegador web.

Capítulo 4: O Sistema de Customizações: Rules, Skills, Plugins & Sidecars

A arquitetura do Antigravity pode ser estendida via **Customizações** salvas globalmente ou por repositório:

```
Estrutura de Locais de Customização:
1. Global Root: C:\Users\<usuario>\.gemini\config\
   ├── rules\       # Diretrizes gerais .md
   ├── skills\      # Pastas de competências técnicas com SKILL.md
   └── plugins\     # Bundles completos de skills e ferramentas
2. Workspace Root: .agents/ (na raiz do repositório)
   ├── rules\
   └── skills\
```

4.1 Estrutura de um Arquivo SKILL.md

Toda skill exige um cabeçalho YAML contendo o nome e a descrição de quando deve ser ativada pelo agente:

```
---
name: deploy-firebase
description: Instruções completas para build, teste e deploy no Firebase Hosting.
---

# Fluxo de Deploy no Firebase Hosting

## Pré-requisitos
- Node.js instalado e dependências atualizadas.

## Passo a Passo
1. Execute a verificação de tipos: npx tsc --noEmit.
2. Gere a build de produção: npm run build.
```

```
3. Valide o diretório dist/.
4. Envie o deploy com o CLI: npx -y firebase-tools@latest deploy --only hosting.
```

Capítulo 5: Model Context Protocol (MCP) & Inspeção em Tempo de Execução

O **Model Context Protocol (MCP)** é o padrão aberto que permite ao Antigravity comunicar-se com ferramentas externas de desenvolvimento:

5.1 Principais Ferramentas MCP Integradas

- **Chrome DevTools MCP:** Inspeciona a árvore DOM, realiza auditorias de acessibilidade (a11y - ARIA, contraste), analisa performance (LCP, INP) e coleta Snapshots de Memória para encontrar vazamentos em aplicações JavaScript.
- **Data Agent Kit MCP:** Conecta-se a bancos de dados GCP (BigQuery, Spanner, GCS) para execução segura de consultas SQL, análise estatística e higienização automática de dados.

Capítulo 6: Arquitetura Multi-Agente: Orquestração Paralela de Subagentes

O Antigravity possui suporte nativo à execução distribuída de **subagentes** via ferramenta `invoke_subagent`:

```
Exemplo de Chamada de Subagente em JSON:
{
  "Subagents": [
    {
      "TypeName": "research",
      "Role": "Pesquisador de Documentação",
      "Prompt": "Inspecione os arquivos da pasta src/hooks e liste todas as chamadas
    }
```

```
    ]
}
```

Desempenho em Paragelo

Subagentes executam em threads isoladas. Você e o agente principal podem continuar trabalhando enquanto o subagente processa pesquisas pesadas ou suítes de testes em segundo plano.

Capítulo 7: Desenvolvimento Autônomo com o Antigravity SDK em Python

Para automação em servidores de CI/CD ou robôs de desenvolvimento não supervisionados, utilize o SDK oficial em Python:

```
from antigravity import AgentSession, TaskGoal

# Criar sessão no repositório
session = AgentSession(workspace_path="./meu-projeto")

# Configurar meta autônoma
goal = TaskGoal(
    instruction="Auditar todos os componentes React, aplicar regras de acessibilidade
    auto_approve_edits=True
)

# Executar meta
result = session.execute(goal)

print(f"Resultado final: {result.status}")
print(f"Arquivos modificados: {len(result.modified_files)}")
```

Capítulo 8: Troubleshooting Avançado, Prevenção de Perda de Dados & Checklist

🚨 Protocolo de Prevenção de Perda de Dados

Comandos que causem destruição irreversível de dados (SQL: `DROP TABLE`, `TRUNCATE`, `DELETE` sem `WHERE`; Cloud: `gsutil rm`, `gcloud projects delete`) exigem obrigatoriamente confirmação explícita do usuário.

8.1 Checklist Definitivo do Desenvolvedor Expert

- [ ] **Inspecionar antes de alterar:** Sempre visualizar arquivos completos antes de reescrever código.
- [ ] **Verificar compilação:** Garantir que `npx tsc --noEmit` passe com zero erros.
- [ ] **Validar build final:** Confirmar que `npm run build` ou `pytest` termine com código de saída 0.
- [ ] **Gravar aprendizado:** Persistir soluções de bugs complexos no repositório com o comando `/learn`.
